

ONEDRAFT

CAD-AI — Aria Powered

Job & Worker Execution System

Version 0.1

Status: Implementation Specification

Architecture: Distributed Asynchronous Execution

Queue: PostgreSQL-backed MVP / Redis-compatible production interface

Workers: API / Model / Geometry / Compliance / Validation / Documentation / Export

Identifiers: UUID

Execution Model: Durable Jobs + Queued Tasks + Worker Claims + State Transitions

AI Layer: Aria Powered

1. EXECUTION DESIGN PRINCIPLE

OneDraft requires a durable execution layer between the Runtime Orchestrator and the computational subsystems.

The Runtime Orchestrator determines:

what must happen

The Job & Worker Execution System determines:

when, where, and under what execution conditions it happens

The workers perform:

the actual computation

The resulting architecture is:

```
CLIENT
↓
API
↓
RUNTIME ORCHESTRATOR
↓
JOB SYSTEM
↓
QUEUE
↓
WORKER
↓
DOMAIN SERVICE
```

↓
RESULT
↓
VALIDATION
↓
PROJECT MODEL

The Job & Worker Execution System therefore becomes the execution backbone of OneDraft.

2. CORE EXECUTION PRINCIPLE

No expensive computational operation shall execute directly inside the synchronous API request lifecycle.

Operations such as:

geometry generation
model regeneration
compliance analysis
validation
drawing generation
sheet composition
document generation
PDF export
package hashing
large-scale model reconstruction
redline resolution

shall execute through durable jobs.

The API returns a `job_id`.

The client may subsequently inspect job state through the API.

3. JOB LIFECYCLE

The universal job lifecycle is:

CREATED
↓
QUEUED
↓
CLAIMED
↓
RUNNING
↓
VALIDATING
↓
COMPLETED

Alternative terminal states are:

FAILED
CANCELLED
EXPIRED
INVALIDATED

A job shall never transition backward from a terminal state.

4. JOB TABLE

system.jobs

Fields:

id UUID PRIMARY KEY
project_id UUID
parent_job_id UUID
job_type VARCHAR(128) NOT NULL
queue_name VARCHAR(128) NOT NULL
priority INTEGER NOT NULL DEFAULT 0
status VARCHAR(32) NOT NULL
requested_by UUID
source_revision_id UUID
source_model_version_id UUID
source_code_manifest_id UUID
payload JSONB NOT NULL
result JSONB
error JSONB
progress INTEGER NOT NULL DEFAULT 0
attempt_count INTEGER NOT NULL DEFAULT 0
max_attempts INTEGER NOT NULL DEFAULT 3
available_at TIMESTAMPTZ NOT NULL
started_at TIMESTAMPTZ
completed_at TIMESTAMPTZ
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL

Indexes:

idx_jobs_project
idx_jobs_status
idx_jobs_queue
idx_jobs_available
idx_jobs_parent
idx_jobs_created

5. JOB TYPES

Initial job types shall include:

MODEL_COMMAND
MODEL_REBUILD
GEOMETRY_GENERATION
GEOMETRY_VALIDATION
COMPLIANCE_ANALYSIS
VALIDATION_RUN
DRAWING_GENERATION
SHEET_COMPOSITION
DOCUMENT_GENERATION
DOCUMENT_HASH
EXPORT
PROJECT_RECONSTRUCTION
REDLINE_RESOLUTION
PACKAGE_VERIFICATION

Additional job types may be introduced without changing the underlying execution architecture.

6. QUEUES

Initial logical queues:

model
geometry
compliance
validation
documentation
export
system

The Runtime Orchestrator assigns jobs to queues according to computational domain.

Example:

ARIA_COMMAND
↓
model
↓
MODEL_MUTATION
↓
geometry
↓
validation
↓
documentation
↓
export

7. JOB PRIORITY

Jobs shall support priority values.

Initial priority classes:

0 = LOW
10 = NORMAL
20 = HIGH
30 = CRITICAL

Higher-priority jobs execute before lower-priority jobs where worker capacity permits.

Priority shall never permit unauthorized bypass of project, authorization, constraint, or validation controls.

8. JOB DEPENDENCIES

Jobs may depend on other jobs.

`system.job_dependencies`

Fields:

`job_id` UUID NOT NULL
`depends_on_job_id` UUID NOT NULL
`dependency_type` VARCHAR(32) NOT NULL
`created_at` TIMESTAMPTZ NOT NULL

Primary key:

`(job_id, depends_on_job_id)`

A dependent job cannot become executable until all required dependencies have reached an acceptable terminal state.

9. DEPENDENCY TYPES

Initial dependency types:

REQUIRED
OPTIONAL

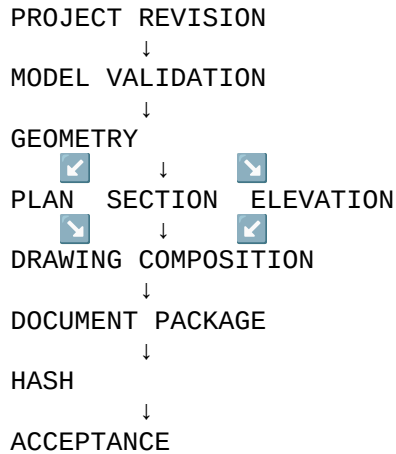
A **REQUIRED** dependency must complete successfully.

An **OPTIONAL** dependency may fail without automatically preventing execution.

10. EXECUTION GRAPH

The Job & Worker Execution System supports directed execution graphs.

Example:



The Runtime Orchestrator creates the execution graph.

Workers execute individual nodes.

11. JOB CLAIMING

Workers shall claim jobs atomically.

Conceptually:

BEGIN

```
SELECT executable job  
FOR UPDATE SKIP LOCKED
```

```
UPDATE job  
SET status = CLAIMED
```

COMMIT

This prevents multiple workers from simultaneously claiming the same job.

12. WORKER REGISTRATION

Workers shall register with the execution subsystem.

system.workers

Fields:

```
id UUID PRIMARY KEY
worker_type VARCHAR(128) NOT NULL
worker_name VARCHAR(255) NOT NULL
host_identifier VARCHAR(255)
version VARCHAR(128)
status VARCHAR(32) NOT NULL
capacity INTEGER NOT NULL DEFAULT 1
last_heartbeat_at TIMESTAMPTZ
metadata JSONB
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

13. WORKER STATES

Workers use:

```
STARTING
IDLE
BUSY
DRAINING
OFFLINE
FAILED
```

A worker that stops sending heartbeats may be considered unavailable.

14. WORKER HEARTBEAT

Workers shall periodically publish:

```
worker_id
status
active_jobs
capacity
timestamp
version
```

The runtime system uses heartbeat state to determine worker availability.

15. WORKER CLAIM RECORD

Every job execution attempt shall be recorded.

```
system.job_attempts
```

Fields:

```
id UUID PRIMARY KEY
job_id UUID NOT NULL
worker_id UUID NOT NULL
attempt_number INTEGER NOT NULL
status VARCHAR(32) NOT NULL
started_at TIMESTAMPTZ
completed_at TIMESTAMPTZ
error JSONB
metrics JSONB
created_at TIMESTAMPTZ NOT NULL
```

Unique:

```
UNIQUE(job_id, attempt_number)
```

This creates an execution history independent of the current job state.

16. HEARTBEAT TIMEOUT

A worker execution may be considered stale when:

```
last_heartbeat_at
```

exceeds the configured worker timeout.

The job is then eligible for recovery.

The system must distinguish:

```
worker failure
```

from:

```
job failure
```

A worker may fail while the underlying job remains recoverable.

17. JOB LEASE

A claimed job receives a lease.

Conceptually:

```
lease_until
```

is associated with the current attempt.

A worker must renew its lease while executing.

If the lease expires, another worker may reclaim the job after recovery safeguards are applied.

18. IDEMPOTENT EXECUTION

Every worker operation shall be designed to tolerate retry.

The job system must assume:

- worker crash
- network interruption
- API timeout
- process restart
- machine failure
- temporary dependency failure

may occur.

A retry must not silently create duplicate project mutations.

19. COMMAND IDEMPOTENCY

For model-changing operations, the execution key shall include:

- project_id
- command_id
- source_revision_id

The Model Service verifies that the command has not already been committed.

This protects the Project Model from duplicate execution.

20. JOB RETRY POLICY

Default retry policy:

maximum_attempts = 3

Retryable conditions include:

- temporary database failure
- worker interruption
- temporary network failure
- temporary storage failure
- service unavailable
- resource exhaustion

Non-retryable conditions include:

- invalid project state
- authorization failure
- constraint violation
- invalid command
- invalid model
- invalid code manifest
- corrupt source data

21. RETRY BACKOFF

Retries shall use exponential backoff.

Conceptually:

- attempt 1 → short delay
- attempt 2 → longer delay
- attempt 3 → final retry

The exact delay remains configuration rather than domain logic.

22. DEAD-LETTER STATE

Jobs exceeding their retry policy enter:

FAILED

with a machine-readable failure record.

Future production deployments may additionally provide:

dead_letter

queue semantics.

23. JOB ERROR MODEL

Errors shall contain:

- code
- message
- category
- retryable
- service
- worker_id
- job_id

details
timestamp

Example:

```
{  
  "code": "GEOMETRY_GENERATION_FAILED",  
  "category": "GEOMETRY",  
  "retryable": true,  
  "service": "geometry-worker",  
  "details": {}  
}
```

24. JOB PROGRESS

Workers shall report progress.

The minimum range is:

0-100

Progress must represent meaningful computational completion rather than arbitrary elapsed time.

Example:

0 = queued
10 = model loaded
30 = geometry generated
60 = geometry validated
80 = drawings regenerated
100 = complete

25. JOB EVENTS

Every significant state transition shall create an event.

`system.job_events`

Fields:

id UUID PRIMARY KEY
job_id UUID NOT NULL
event_type VARCHAR(128) NOT NULL
status VARCHAR(32)
payload JSONB NOT NULL
timestamp TIMESTAMPTZ NOT NULL

The table is append-only.

26. JOB STATE MACHINE

Permitted transitions:

CREATED → QUEUED

QUEUED → CLAIMED

CLAIMED → RUNNING

RUNNING → VALIDATING

RUNNING → COMPLETED

VALIDATING → COMPLETED

RUNNING → FAILED

VALIDATING → FAILED

QUEUED → CANCELLED

CLAIMED → CANCELLED

RUNNING → INVALIDATED

Invalid state transitions must be rejected.

27. CANCELLATION

A queued job may be cancelled.

A running job may receive a cancellation request.

Workers must periodically check cancellation state.

Cancellation shall not automatically reverse mutations that have already been committed.

Model rollback remains a separate revision operation.

28. MODEL JOB

A model job executes a domain command.

MODEL_COMMAND

Input:

command_id

project_id

revision_id
model_version_id

The worker passes the command to:

Project Model Service

The Project Model Service remains the authoritative boundary for model mutation.

The worker must never directly manipulate database tables outside defined domain services.

29. GEOMETRY JOB

A geometry job operates against a specific model version.

GEOMETRY_GENERATION

Input:

project_id
model_version_id
element_ids
geometry_version

The geometry worker generates or regenerates geometry.

Before committing the result, it verifies that the source model version remains valid.

30. GEOMETRY STALENESS PROTECTION

Every geometry job shall record:

project_id
model_version_id
element_id
geometry_version

If the model changes before geometry completion, the worker shall not silently promote stale geometry to current state.

The job may instead become:

INVALIDATED

and a replacement geometry job may be scheduled.

31. COMPLIANCE JOB

A compliance job executes against an immutable regulatory manifest.

COMPLIANCE_ANALYSIS

Input:

project_id
revision_id
model_version_id
code_manifest_id

The worker evaluates the project against the rules contained in the selected manifest.

The result becomes part of the validation and provenance chain.

32. VALIDATION JOB

VALIDATION_RUN

Input:

project_id
revision_id
model_version_id
code_manifest_id
validation_categories

The validation worker executes:

CODE
SPATIAL
ACCESSIBILITY
DOCUMENT
COORDINATION

as applicable.

Results are stored through the Validation Service.

33. DRAWING GENERATION JOB

DRAWING_GENERATION

Input:

project_id
revision_id

model_version_id
view_ids
drawing_types

The worker generates drawing references and associated graphical content.

The drawing must remain tied to the exact:

project
revision
model_version
view

from which it was generated.

34. SHEET COMPOSITION JOB

SHEET_COMPOSITION

The sheet compositor assembles:

views
drawings
dimensions
annotations
notes
schedules
title blocks
revision information

onto the appropriate sheet template.

For architectural output, the initial supported paper configuration includes:

ARCH_D

Additional paper sizes may be added later.

35. DOCUMENT GENERATION JOB

DOCUMENT_GENERATION

The document worker consumes validated drawing and documentation outputs.

The package shall retain references to:

project_id
revision_id
model_version_id
code_manifest_id

validation_run_id

The generated package is therefore reproducible.

36. DOCUMENT HASH JOB

DOCUMENT_HASH

The hashing worker computes a cryptographic hash of the completed document package.

The hash becomes part of the package provenance record.

Example:

SHA-256

The exact algorithm may be configuration-driven, but SHA-256 is the initial implementation target.

37. PACKAGE VERIFICATION JOB

PACKAGE_VERIFICATION

The verification worker confirms:

```
document exists
expected files exist
file hashes match
revision matches
model version matches
code manifest matches
validation record exists
required drawings exist
required schedules exist
```

Only a successfully verified package may proceed to final acceptance.

38. PROJECT RECONSTRUCTION JOB

PROJECT_RECONSTRUCTION

This job reconstructs a project state from:

```
project_id
revision_id
model_version_id
```


The reconstruction system verifies that historical state can be recreated from the persisted model and revision records.

This directly implements the OneDraft reconstruction requirement.

39. REDLINE RESOLUTION JOB

REDLINE_RESOLUTION

The job consumes:

redline_id
revision_id
model_version_id

The worker passes the requested change through the Runtime Orchestrator and Project Model Service.

A redline cannot be marked:

RESOLVED

until the associated model operation has successfully completed and required validation has passed.

40. JOB RESULT MODEL

Successful jobs shall return a standardized result structure.

```
{
  "status": "COMPLETED",
  "job_id": "uuid",
  "project_id": "uuid",
  "revision_id": "uuid",
  "model_version_id": "uuid",
  "result": {},
  "metrics": {}
}
```

The result payload shall contain references rather than unnecessarily duplicating large computational objects.

41. JOB METRICS

Workers shall record execution metrics.

Initial metrics include:

```
queue_wait_ms
execution_ms
validation_ms
memory_usage
cpu_time
input_size
output_size
retry_count
```

Metrics support:

```
performance optimization
capacity planning
failure analysis
cost analysis
worker scaling
```

42. RESOURCE LIMITS

Jobs may specify resource requirements.

Example:

```
cpu
memory
storage
gpu
timeout
```

Example:

```
{
  "cpu": 2,
  "memory_mb": 4096,
  "gpu": false,
  "timeout_seconds": 900
}
```

The scheduler uses these requirements to select an appropriate worker.

43. WORKER CAPABILITY MODEL

Workers may advertise capabilities.

Example:

```
geometry-worker:
  geometry
  spatial-validation
```

documentation-worker:
drawing-generation
sheet-composition

export-worker:
pdf
package-generation

The scheduler shall only assign jobs to workers capable of executing the requested job type.

44. WORKER VERSION COMPATIBILITY

Each worker reports:

worker_version
domain_version

Jobs may specify minimum compatible versions.

This prevents incompatible worker implementations from consuming jobs created against newer domain contracts.

45. ARTIFACT REFERENCES

Large job outputs shall not be embedded directly into PostgreSQL JSONB payloads.

Instead, the job result may contain:

artifact_id
storage_reference
content_hash
content_type
size

The object-storage layer becomes responsible for large binary artifacts.

46. JOB INPUT IMMUTABILITY

Once a job reaches:

CLAIMED

its computational input shall be immutable.

If the underlying project state changes, the existing job is either:

completed against its recorded source state

or:

invalidated

A running job shall never silently switch to a newer model version.

47. SOURCE STATE LOCK

Jobs that modify the Project Model shall identify:

`expected_revision`

`expected_model_version`

The domain service verifies those values before committing changes.

If the expected state is no longer current:

`STALE_COMMAND`

is returned.

The command is not applied against the newer state without explicit orchestration.

48. CONCURRENCY CONTROL

OneDraft shall use optimistic concurrency control for project model mutations.

The authoritative comparison is:

`expected_revision`

against:

`current_revision`

A mismatch prevents an unsafe mutation.

49. JOB TRANSACTION BOUNDARY

The job lifecycle itself and the domain transaction remain distinct.

Conceptually:

`JOB CLAIM`

↓
WORKER EXECUTION
↓
DOMAIN TRANSACTION
↓
DOMAIN RESULT
↓
JOB COMPLETION

A failed domain transaction shall not be represented as a successful job.

50. JOB COMPLETION

A job may transition to:

COMPLETED

only when:

required computation completed
required domain transaction committed
required validation completed
required result persisted

have all succeeded.

51. PARTIAL SUCCESS

Where a job contains multiple independent operations, the result may identify:

completed_items
failed_items
skipped_items

However, partial success shall never be interpreted as complete project success unless the job contract explicitly permits partial completion.

52. JOB OBSERVABILITY

The execution layer shall expose:

job status
worker status
queue depth
active jobs
failed jobs

retry count
execution duration

to administrative and operational interfaces.

53. API JOB ENDPOINT

The API shall expose:

GET /api/v1/jobs/{job_id}

Response:

```
{
  "data": {
    "job_id": "uuid",
    "type": "DOCUMENT_GENERATION",
    "status": "RUNNING",
    "progress": 72,
    "created_at": "ISO-8601",
    "started_at": "ISO-8601"
  },
  "metadata": {
    "request_id": "uuid",
    "timestamp": "ISO-8601"
  }
}
```

54. JOB LIST ENDPOINT

The API shall expose:

GET /api/v1/projects/{project_id}/jobs

Supported filters:

status
job_type
created_after
created_before

The endpoint shall use the standard OneDraft pagination model.

55. JOB CANCELLATION ENDPOINT

The API shall expose:

POST /api/v1/jobs/{job_id}/cancel

Response:

```
{
  "data": {
    "job_id": "uuid",
    "status": "CANCELLED"
  }
}
```

Cancellation must respect authorization and project ownership.

56. JOB CREATION ENDPOINT

Internal services may create jobs through:

POST /internal/jobs

The public API does not need to expose arbitrary job creation.

The Runtime Orchestrator remains responsible for constructing valid execution requests.

57. QUEUE ADAPTER

The application shall define an abstract queue interface.

Conceptually:

```
QueueAdapter
  enqueue()
  claim()
  acknowledge()
  reject()
  retry()
  cancel()
```

The domain system must not depend directly on a particular queue implementation.

58. MVP QUEUE IMPLEMENTATION

The MVP may use PostgreSQL as the durable queue mechanism.

The implementation may use:

FOR UPDATE SKIP LOCKED

for job claiming.

This minimizes infrastructure requirements during initial development.

59. PRODUCTION QUEUE IMPLEMENTATION

The architecture shall permit migration to a dedicated queue system such as:

Redis
RabbitMQ
NATS
Kafka

without changing the OneDraft domain contracts.

The queue adapter provides the abstraction boundary.

60. QUEUE DURABILITY

A queued job must survive:

API restart
worker restart
application restart
machine restart

The job record is therefore persisted independently of the worker process.

61. ORCHESTRATOR / WORKER SEPARATION

The Runtime Orchestrator does not perform computational work directly.

It creates:

job graph

and delegates execution to workers.

Example:

RUNTIME ORCHESTRATOR


```
create MODEL_COMMAND job
create GEOMETRY_GENERATION dependency
create VALIDATION_RUN dependency
create DRAWING_GENERATION dependency
create DOCUMENT_GENERATION dependency
```

Workers then execute the graph.

62. EVENT-DRIVEN CONTINUATION

Job completion may trigger downstream jobs.

Example:

```
GEOMETRY_GENERATION
  ↓
GEOMETRY_COMPLETED
  ↓
VALIDATION_RUN CREATED
```

Similarly:

```
VALIDATION_COMPLETED
  ↓
DRAWING_GENERATION CREATED
```

And:

```
DRAWING_GENERATION COMPLETED
  ↓
DOCUMENT_GENERATION CREATED
```

The Runtime Orchestrator remains responsible for determining whether the next stage is valid.

63. FAILURE PROPAGATION

If a required upstream job fails:

```
UPSTREAM FAILED
  ↓
DEPENDENT JOBS
  ↓
BLOCKED
```

Blocked jobs shall not execute against incomplete prerequisites.

They may be marked:

```
CANCELLED
```

or:

INVALIDATED

according to orchestration policy.

64. VALIDATION GATE

Certain downstream operations require successful validation.

For example:

VALIDATION

↓

DOCUMENT GENERATION

The document generation job shall verify that the required validation state exists before proceeding.

This prevents generation of a final package from an unvalidated model state.

65. FINAL PACKAGE GATE

Final document packaging requires:

model state valid
geometry state valid
required drawings complete
required validation complete
code manifest identified
document artifacts complete

Only then may:

DOCUMENT_HASH

and:

PACKAGE_VERIFICATION

execute.

66. ACCEPTANCE GATE

Customer acceptance occurs only after:

PACKAGE_VERIFICATION = PASSED

The acceptance subsystem records:

```
project_id  
revision_id  
model_version_id  
document_package_id  
code_manifest_id  
validation_run_id  
customer_id  
accepted_at
```

67. AUDIT INTEGRATION

Every significant execution event shall integrate with:

```
audit.project_events
```

Examples:

```
JOB_CREATED  
JOB_CLAIMED  
JOB_STARTED  
JOB_RETRIED  
JOB_FAILED  
JOB_COMPLETED  
JOB_CANCELLED  
JOB_INVALIDATED  
WORKER_FAILURE  
WORKER_RECOVERY
```

The audit record must preserve the relationship between execution state and project state.

68. SECURITY BOUNDARY

Workers shall not receive unrestricted database credentials.

Workers interact through:

```
domain services  
application services  
controlled repositories
```

The execution layer therefore maintains the existing OneDraft security boundary:

```
ARIA  
↓  
ORCHESTRATOR  
↓  
JOB  
↓
```

DOMAIN SERVICE
↓
DATABASE

69. AI EXECUTION BOUNDARY

Aria may request computational work.

Aria does not directly execute arbitrary worker code.

The workflow is:

```
USER REQUEST
↓
ARIA
↓
COMMAND
↓
RUNTIME ORCHESTRATOR
↓
JOB
↓
AUTHORIZED WORKER
↓
DOMAIN SERVICE
```

This maintains the AI command safety boundary established by the OneDraft architecture.

70. ARIA JOB CREATION

When Aria determines that a task requires asynchronous computation, it creates an execution request containing:

```
project_id
revision_id
model_version_id
operation
target_objects
parameters
constraints
validation_requirements
```

The Runtime Orchestrator converts that request into one or more jobs.

71. EXAMPLE ARIA EXECUTION

User request:

Move the rear bedroom wall 300 mm north while keeping the exterior envelope unchanged.

Aria determines:

MOVE_ELEMENT

The Runtime Orchestrator creates:

MODEL_COMMAND

which creates:

GEOMETRY_GENERATION

followed by:

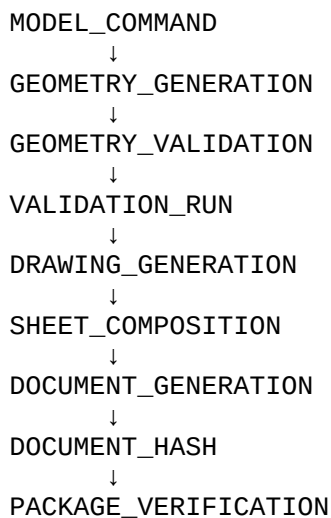
VALIDATION_RUN

followed by:

DRAWING_GENERATION

if affected documentation requires regeneration.

72. EXECUTION GRAPH EXAMPLE



The exact graph is determined by the Runtime Orchestrator according to the operation.

73. JOB GRAPH IDENTIFIER

Each coordinated execution chain may receive:

execution_id UUID

The execution ID groups related jobs belonging to the same user-requested operation.

Example:

```
execution_id
  ↓
MODEL_COMMAND
  ↓
GEOMETRY
  ↓
VALIDATION
  ↓
DRAWINGS
  ↓
DOCUMENTS
```

This provides a higher-level execution trace than the individual job IDs.

74. EXECUTION RECORD

system.executions

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
requested_by UUID
operation VARCHAR(128) NOT NULL
status VARCHAR(32) NOT NULL
source_revision_id UUID
source_model_version_id UUID
root_job_id UUID
result JSONB
error JSONB
created_at TIMESTAMPTZ NOT NULL
started_at TIMESTAMPTZ
completed_at TIMESTAMPTZ
```

The execution record represents the user-visible operation.

75. EXECUTION STATES

Initial execution states:

CREATED
RUNNING
VALIDATING
COMPLETED
FAILED
CANCELLED
INVALIDATED

The execution becomes **COMPLETED** only when all required jobs have completed successfully.

76. EXECUTION / JOB RELATIONSHIP

The relationship is:

ONE EXECUTION
↓
MANY JOBS

A job may belong to one execution.

A child job may additionally reference:

parent_job_id

This creates both:

execution-level grouping

and:

job-level dependency

77. WORKER LOGGING

Worker logs must include:

timestamp
worker_id
job_id
execution_id
project_id
revision_id
model_version_id
severity
message
metadata

This permits complete tracing of a computational operation.

78. STRUCTURED LOGGING

Worker logs shall use structured machine-readable records.

Example:

```
{
  "level": "INFO",
  "worker": "geometry-worker",
  "job_id": "uuid",
  "project_id": "uuid",
  "model_version_id": "uuid",
  "message": "Geometry generation completed"
}
```

Free-form logs may exist for debugging but shall not replace structured operational events.

79. TRACE CORRELATION

The following identifiers shall be propagated through the execution stack:

```
request_id
execution_id
job_id
project_id
revision_id
model_version_id
```

This allows a single user request to be traced from:

```
API
↓
ARIA
↓
ORCHESTRATOR
↓
JOB
↓
WORKER
↓
DOMAIN SERVICE
↓
DATABASE
```

80. TIMEOUT POLICY

Jobs may specify execution timeouts.

Timeouts are divided into:

queue timeout
execution timeout
validation timeout

A timeout produces a structured failure state.

The job may be retried when the failure is classified as recoverable.

81. RESOURCE EXHAUSTION

Workers shall detect:

memory exhaustion
CPU exhaustion
storage exhaustion
GPU exhaustion
external service exhaustion

Resource exhaustion shall be recorded separately from logical domain failure.

This distinction allows infrastructure scaling rather than unnecessary model retries.

82. WORKER DRAINING

A worker may enter:

DRAINING

The worker stops accepting new jobs while completing active jobs.

This permits safe deployment and maintenance.

83. WORKER SHUTDOWN

Graceful shutdown sequence:

DRAIN
↓
STOP CLAIMING
↓
COMPLETE ACTIVE JOBS
↓
ACKNOWLEDGE JOBS
↓
OFFLINE

If graceful completion is impossible, leases allow another worker to recover the affected jobs.

84. JOB RECOVERY

Recovery process:

```
detect stale lease
↓
verify worker unavailable
↓
record worker failure
↓
release job
↓
increment attempt
↓
apply retry policy
↓
requeue
```

Recovery must preserve the original execution history.

85. DATABASE CONSISTENCY

Job state changes shall be transactional.

When a job completion causes a domain-state mutation, the application shall ensure that the domain result and corresponding job state cannot become misleadingly divergent.

Where distributed transaction semantics are impractical, the system shall use durable event records and reconciliation workers.

86. RECONCILIATION WORKER

The system shall include a:

system-reconciliation-worker

responsible for detecting inconsistencies such as:

```
job marked completed without result
artifact exists without completion record
worker lease expired
execution incomplete despite all jobs completed
dependent job blocked incorrectly
document package missing expected artifact
```

The reconciliation worker does not silently modify domain state without recording an audit event.

87. SCHEDULER

The scheduler is responsible for selecting executable work.

Conceptually:

```
QUEUED JOBS
↓
FILTER DEPENDENCIES
↓
FILTER RESOURCE REQUIREMENTS
↓
FILTER WORKER CAPABILITIES
↓
FILTER PRIORITY
↓
CLAIM JOB
```

The scheduler must never bypass dependency or authorization requirements.

88. BACKPRESSURE

When computational demand exceeds worker capacity, the queue shall absorb the workload.

Example:

```
100 drawing jobs
20 available workers
```

The system retains:

```
80 queued jobs
```

rather than overwhelming the application layer.

89. RATE LIMITING

The execution system may limit:

```
jobs per project
jobs per organization
jobs per user
jobs per worker
```

This prevents a single workload from monopolizing the execution system.

Limits shall be configurable.

90. FAIRNESS

Where multiple organizations share infrastructure, scheduling should prevent sustained starvation of lower-priority tenants.

The initial MVP may use priority plus FIFO ordering.

Future production scheduling may implement weighted fairness.

91. JOB RETENTION

Completed job records shall remain available for operational and audit purposes.

Retention policy may distinguish:

- active jobs
- completed jobs
- failed jobs
- audit events
- execution metrics

Historical project execution records shall not be deleted merely because their associated job has aged beyond operational retention.

92. EXECUTION RECONSTRUCTION

Given:

execution_id

the system shall be able to reconstruct:

- request
 - ↓
- root job
 - ↓
- child jobs
 - ↓
- worker attempts
 - ↓
- results
 - ↓

project revision
↓
generated artifacts

This becomes part of the OneDraft provenance architecture.

93. FAILURE RECONSTRUCTION

Given a failed execution, the system shall be able to identify:

which job failed
which worker executed it
which attempt failed
what input state was used
what error occurred
whether the error was retryable
what downstream jobs were blocked

94. TESTING REQUIREMENTS

The Job & Worker Execution System shall be tested for:

job creation
job claiming
concurrent claiming
dependency enforcement
retry
backoff
cancellation
worker failure
lease expiration
job recovery
duplicate execution
stale model detection
resource failure
successful completion
partial failure
execution reconstruction

95. FIRST JOB SYSTEM INTEGRATION TEST

The first integration test shall execute:

1. Create project

2. Create model version
3. Create revision
4. Create MODEL_COMMAND job
5. Queue job
6. Claim job
7. Execute model mutation
8. Create GEOMETRY_GENERATION job
9. Generate geometry
10. Validate geometry
11. Create VALIDATION_RUN job
12. Execute validation
13. Create DRAWING_GENERATION job
14. Generate drawings
15. Create DOCUMENT_GENERATION job
16. Generate package
17. Hash package
18. Verify package
19. Complete execution
20. Retrieve complete execution history

The entire operation must remain traceable through:

execution_id
job_id
revision_id
model_version_id

96. INITIAL WORKER SERVICES

The first implementation shall provide:

model-worker
geometry-worker
compliance-worker
validation-worker
documentation-worker

export-worker
system-worker

Each worker should share a common worker runtime while implementing domain-specific handlers.

97. COMMON WORKER RUNTIME

The worker runtime shall provide:

registration
heartbeat
job polling
job claiming
lease renewal
execution
progress reporting
error handling
retry handling
logging
metrics
shutdown

Domain workers should not duplicate these mechanisms.

98. DOMAIN HANDLER INTERFACE

Each worker shall expose a handler conceptually equivalent to:

`execute(job) -> JobResult`

The handler receives a validated job envelope.

The handler does not determine whether the job should exist.

That decision belongs to the Runtime Orchestrator.

99. JOB ENVELOPE

The internal job envelope shall contain:

job_id
execution_id
project_id
job_type
revision_id
model_version_id
code_manifest_id

payload
attempt_number

The envelope is immutable for the duration of an attempt.

100. WORKER RESULT

A worker returns:

status
result
artifacts
metrics
events

Example:

```
{  
  "status": "COMPLETED",  
  "result": {  
    "affected_objects": []  
  },  
  "artifacts": [],  
  "metrics": {}  
}
```

The common runtime converts this into the persistent job completion state.

101. IMPLEMENTATION BOUNDARY

The Job & Worker Execution System shall not redefine:

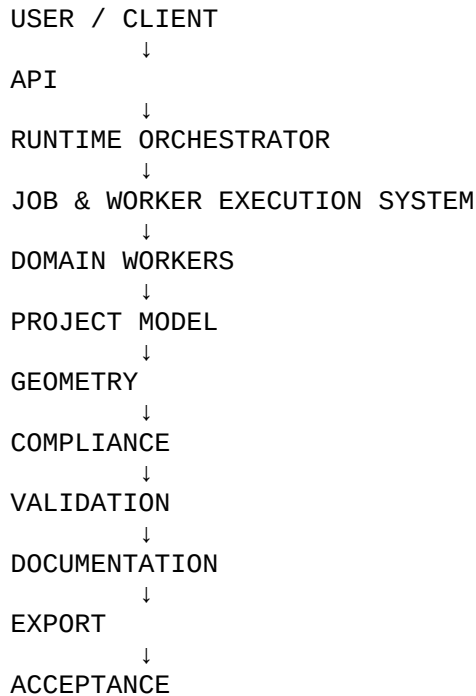
Project Model
Geometry Model
Compliance Model
Validation Model
Revision Model
Documentation Model
Acceptance Model

Those systems remain authoritative within their existing domains.

The Job & Worker Execution System coordinates their execution.

102. ARCHITECTURAL POSITION

The resulting OneDraft stack is now:



The Job & Worker layer therefore connects the previously established domain architecture into an executable distributed system.

103. MVP SUCCESS CRITERIA

The Job & Worker Execution System is successful when it can:

Create jobs

Queue jobs

Claim jobs atomically

Execute jobs

Track progress

Record attempts

Retry recoverable failures

Reject non-retryable failures

Recover from worker failure

Enforce job dependencies

Prevent duplicate model mutations

Detect stale model versions

Execute domain workers

Generate downstream jobs

Track complete execution graphs

Record structured events

Expose job status through the API

Reconstruct execution history

104. FIRST EXECUTABLE ARTIFACTS

The next implementation package shall consist of:

`system_jobs.sql`

The PostgreSQL migration establishing:

`jobs`
`job_dependencies`
`job_attempts`
`job_events`
`workers`
`executions`

and associated indexes and constraints.

`job_types.py`

The Python/Pydantic definitions for:

`Job`
`JobStatus`
`JobType`
`JobAttempt`
`JobEvent`
`Execution`
`ExecutionStatus`
`Worker`

`queue_adapter.py`

The queue abstraction supporting:

`enqueue`
`claim`

acknowledge
reject
retry
cancel

worker_runtime.py

The common worker lifecycle implementation supporting:

registration
heartbeat
claim
lease
execution
progress
completion
failure
retry
shutdown

job_orchestrator.py

The Runtime Orchestrator integration responsible for constructing execution graphs.

105. STACK INTEGRATION

The implementation shall connect the Job & Worker Execution System to the existing OneDraft repository architecture:

apps/
 api/
 worker/
 web/

packages/
 domain/
 geometry/
 compliance/
 validation/
 documentation/
 orchestration/
 jobs/

The exact package placement may evolve during implementation, but the domain boundaries shall remain explicit.

106. FINAL ARCHITECTURAL DECISION

OneDraft shall not depend upon synchronous execution for computationally expensive operations.

The platform shall operate as:

```
REQUEST
↓
COMMAND
↓
ORCHESTRATION
↓
EXECUTION GRAPH
↓
DURABLE JOBS
↓
SPECIALIZED WORKERS
↓
DOMAIN SERVICES
↓
VALIDATION
↓
DERIVED DOCUMENTATION
```

This allows OneDraft to scale from a development workstation to distributed production infrastructure without changing the fundamental project model.

107. SPECIFICATION STATUS

ONEDRAFT JOB & WORKER EXECUTION SYSTEM v0.1

STATUS: ESTABLISHED

The OneDraft architecture now contains:

- Product Definition
- System Architecture
- Technology Architecture
- Domain Model
- Persistence Model
- API Contract
- Geometry & Parametric Model Engine
- Compliance & Regulatory Engine
- Validation Engine
- Documentation & Drawing Generation Engine
- Runtime Orchestrator
- Job & Worker Execution System
- Revision Architecture
- Redline Architecture
- Document Provenance
- Customer Acceptance
- Audit Architecture

The next implementation stage is the creation of the executable Job & Worker foundation:

`system_jobs.sql`

`job_types.py`

`queue_adapter.py`

`worker_runtime.py`

`job_orchestrator.py`

These artifacts establish the durable execution layer required for OneDraft to move from a collection of domain subsystems into an operational computational platform.